香港城市大學
City University of Hong Kong
Innovating into the Future

CS5493: Tutorial 7

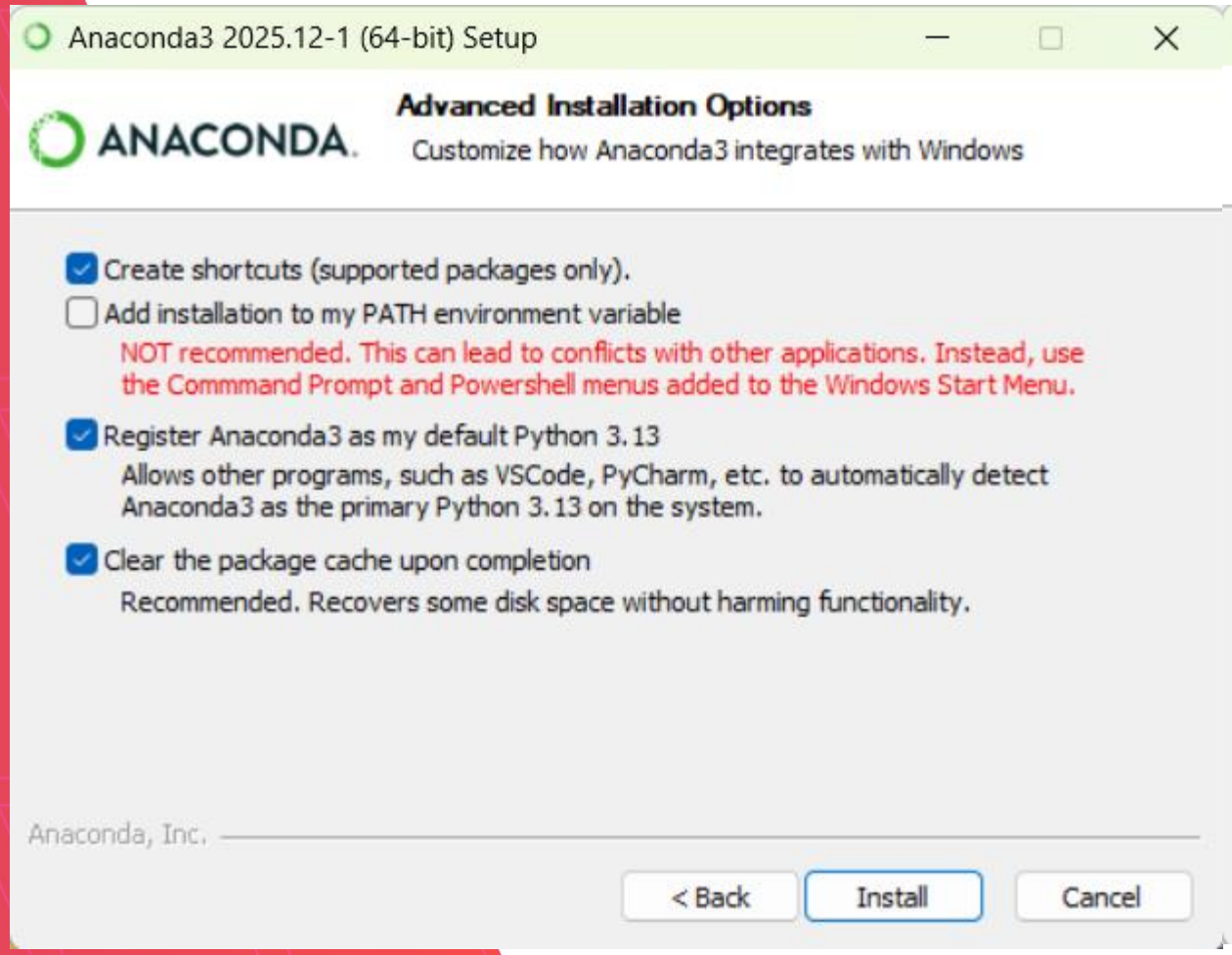
Environment Perception with Camera Image

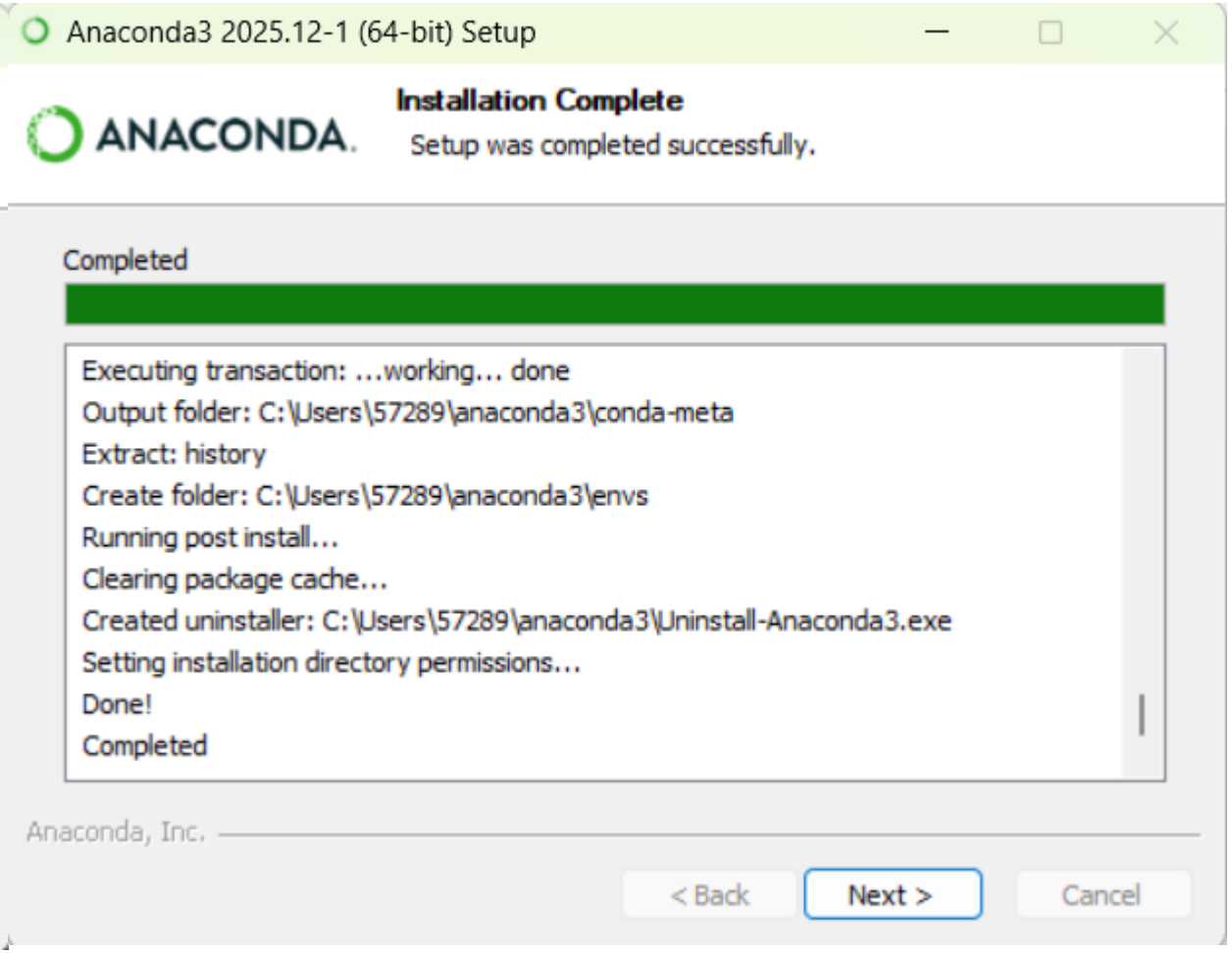
Pre-tutorial Preparation

This tutorial is based on jupyter notebook similar to Assignment 1, which requires setting up the environment for Python.
We usually use Anaconda/Miniconda/Miniforge, but you can also set up python and VSCode along with jupyter packages in your own environment!

Anaconda/Miniconda: https://anaconda.org/
Miniforge: https://github.com/conda-forge/miniforge

Pre-tutorial Preparation

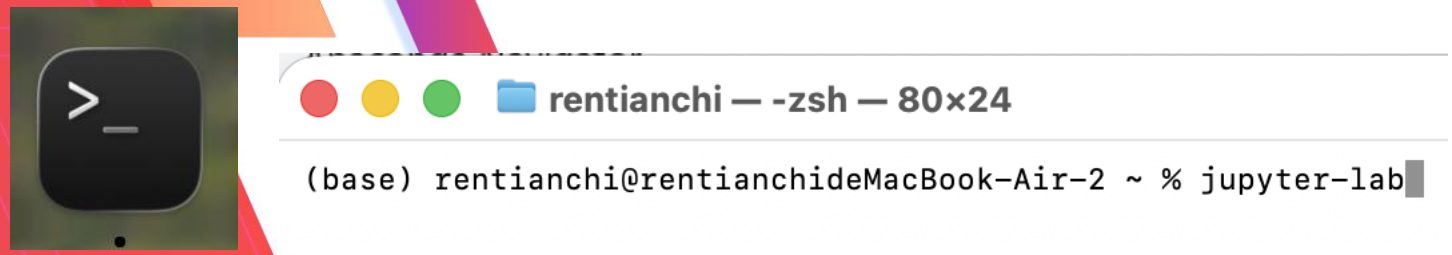




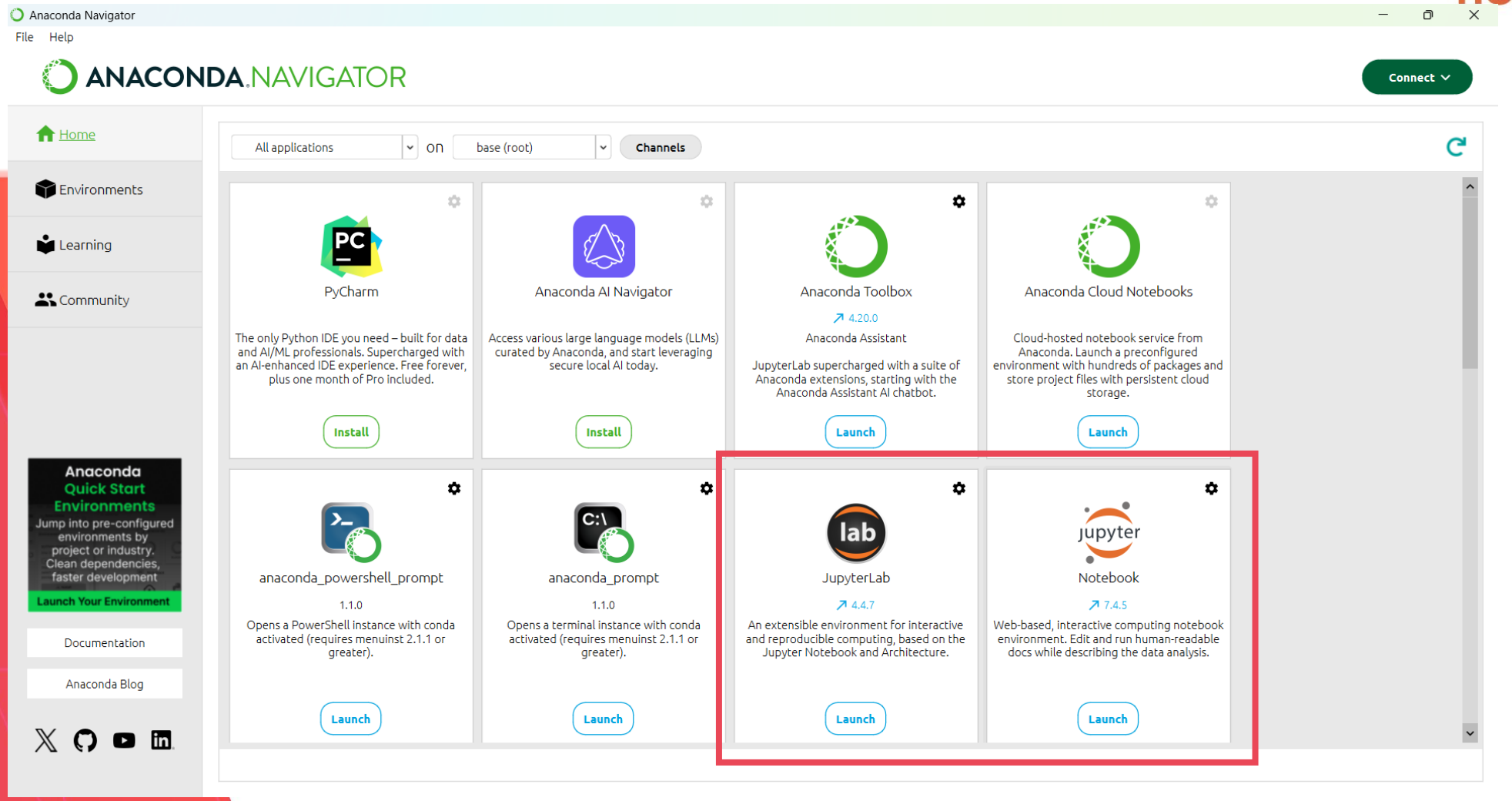
Pre-tutorial Preparation

Windows: On Start Page you may find Anaconda Navigator

MacOS: Terminal -> jupyter-lab



Pre-tutorial Preparation



JupyterLab and Jupyter Notebook both offer access to .ipynb file

Pre-tutorial Preparation

Run or stop the code blocks

Select your kernel

Tutorial4_fi... - JupyterLab

File Edit View Run Kernel Tabs Settings Help

Launcher | Tutorial4_fix.ipynb

Markdown | Notebook | Python 3 (ipykernel)

/ Tutorial4_VisualPerceptionOverall / T

Name	Modified
_pptx_extract	33m ago
data	33m ago
images	33m ago
Slides_latex	33m ago
m6bk.py	2mo ago
Tutorial4_fix_bla...	2mo ago
Tutorial4_fix.html	2mo ago
Tutorial4_fix.ipynb	last mo.
Tutorial4_raw.ipy...	5mo ago
Tutorial4.zip	2mo ago
Tutorial07.pptx	8d ago

Environment Perception For Self-Driving Cars

In this tutorial, you will:

- Use the output of semantic segmentation neural networks to implement drivable space estimation in 3D.
- Use the output of semantic segmentation to filter errors in the output of 2D object detectors.
- Use the filtered 2D object detection results to determine how far obstacles are from the self-driving car.

For most exercises, you are provided with a suggested outline. You are encouraged to diverge from the outline if you think there is a better, more efficient way to solve a problem.

Please go through cells in order. Lower cells will depend on higher cells to work properly.

Despite packages loaded below, you are also encouraged to use other packages or write your own function. Run the cell bellow to import the required packages:

```
[ ]: # This block is used to install the necessary packages and should be run only once.
!pip3 install numpy
!pip3 install opencv-python
!pip3 install matplotlib
!pip3 install scipy
!pip3 install scikit-learn
!pip3 install scikit-image
!pip3 install scikit-learn
!pip3 install matplotlib
```

```
Requirement already satisfied: numpy in /opt/miniconda3/lib/python3.13/site-packages (2.2.6)
Requirement already satisfied: opencv-python in /opt/miniconda3/lib/python3.13/site-packages (4.12.0.88)
```

Would you like to get notified about official Jupyter news?

Open privacy policy | Yes | No

Simple | 1 | Python 3 (ipykernel) | Idle | Mode: Command | Ln 1, Col 1 | Tutorial4_fix.ipynb | 1

Data Loading

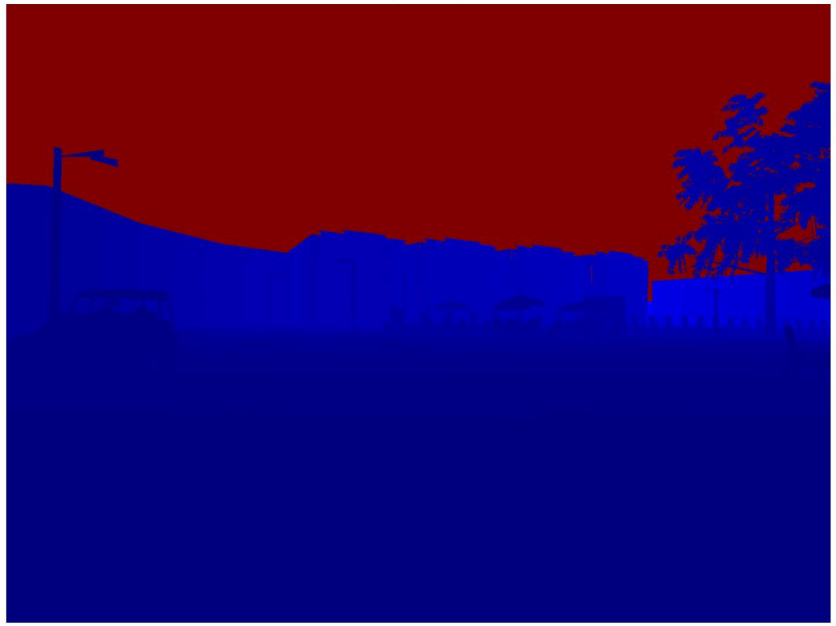
Convenient dataset handler class to read and iterate through samples from CARLA simulator Contains three test data frames: 0, 1, and 2

Each frame contains:

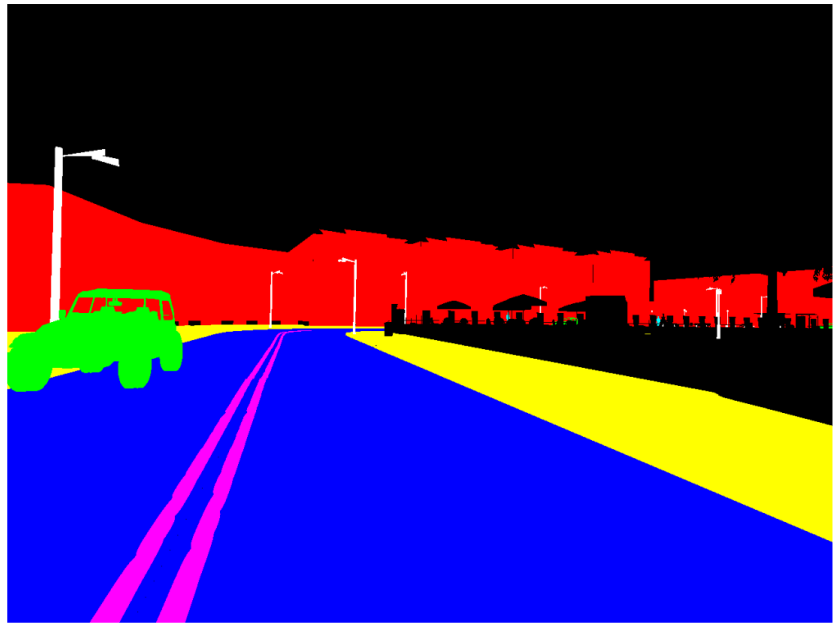
- RGB camera image
- Depth image (depth in meters per pixel)
- Segmentation output (category per pixel)
- Object detection output (bounding boxes)

RGB Original

Depth Image



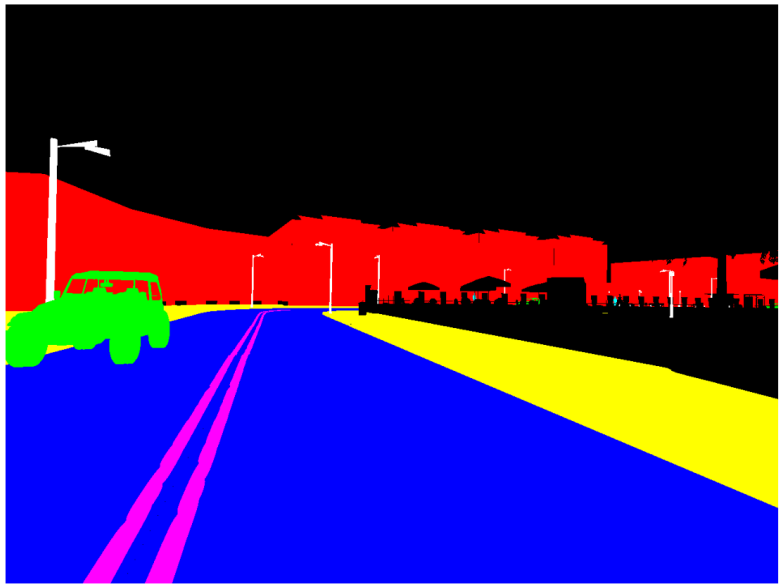
Semantic Segmentation



Data Loading

Segmentation Category Mappings

Category	Index	Visualization Color
Background	0	Black
Buildings	1	Red
Pedestrians	4	Teal
Poles	5	White
Lane Markings	6	Purple
Roads	7	Blue
Side Walks	8	Yellow
Vehicles	10	Green



Task 1.
Drivable Space Estimation

Drivable Space Estimation

Goal: Estimate drivable space in 3D using semantic segmentation output

Steps:

1. Estimate x, y, z coordinates of every pixel
2. Extract road pixels from segmentation
3. Use RANSAC to estimate ground plane
4. Compute drivable space mask

Drivable Space Estimation

Coordinate conversion

Given a depth map and a calibration matrix K:

$$K = \begin{bmatrix} f & 0 & c_u \\ 0 & f & c_v \\ 0 & 0 & 1 \end{bmatrix}$$

Compute the 3-D coordinates:

$$\begin{cases} z = depth \\ x = \dfrac{z(u - c_u)}{f} \\ y = \dfrac{z(v - c_v)}{f} \end{cases}$$

Where (u, v) are pixel coordinates in the image.

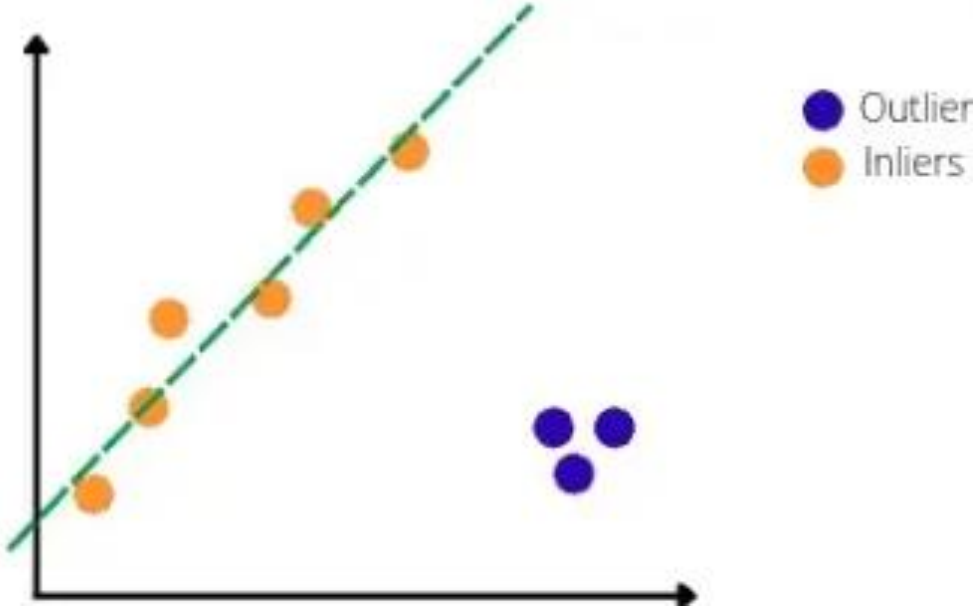
Drivable Space Estimation

RANSAC for Drivable Plane Estimation

RANSAC algorithm can effectively filter the outliers and determine the plane (ground in this scenario)

RANSAC Algorithm:

- Randomly select 3 points from road pixels
- Compute plane model: ax +by +cz +d = 0
- Calculate distance from plane to all points
- Count inliers (distance < threshold)
- Keep the plane with most inliers
- Repeat for maximum iterations or until enough inliers
- Re-estimate plane using all inliers



Drivable Space Estimation

RANSAC for Drivable Plane Estimation

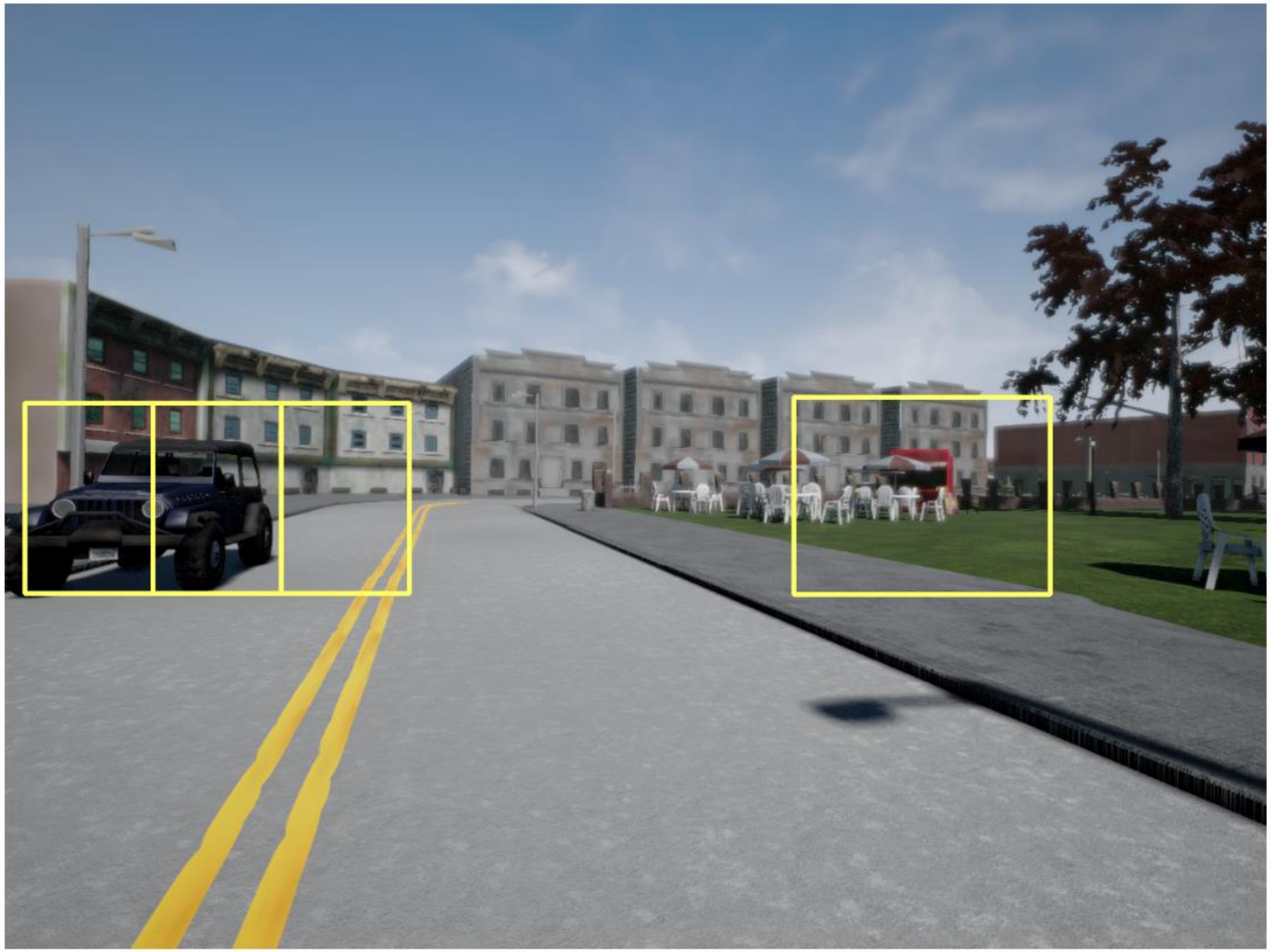
Groundtruth Plane

Estimated Plane

Task 2.
Object Detection

Object Detection

2D Object Detection Output



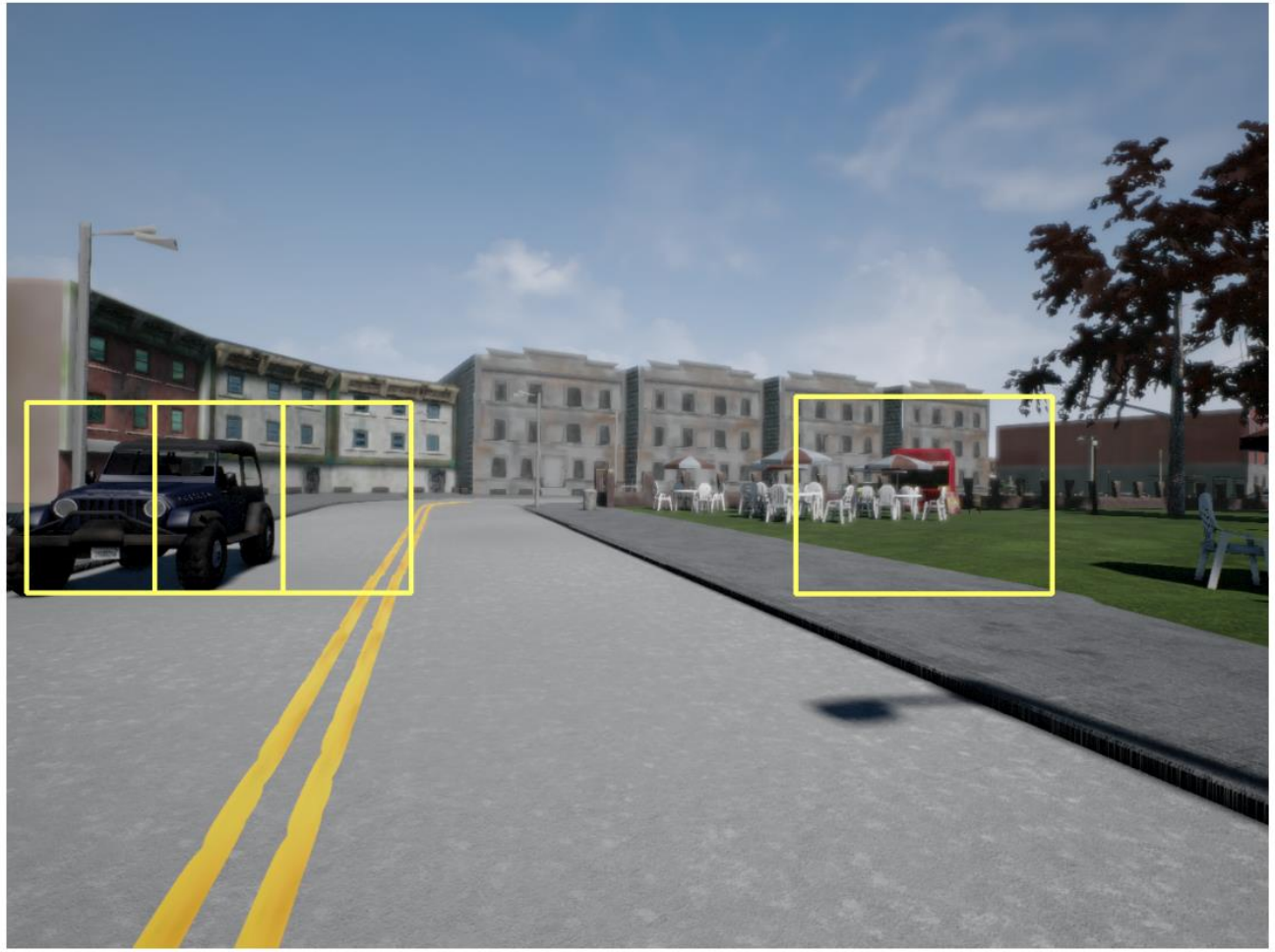
Object Detection

Each detection has format:
[category, xmin, ymin,xmax,ymax,score]

- Category: ’Car’, ’Pedestrian’, or ’Cyclist’
- (xmin, ymin): Top-left corner
- (xmax,ymax): Bottom-right corner
- Score: Neural network confidence (softmax output)

Problem: High recall detector produces false positives despite high scores!

2D Object Detection Output



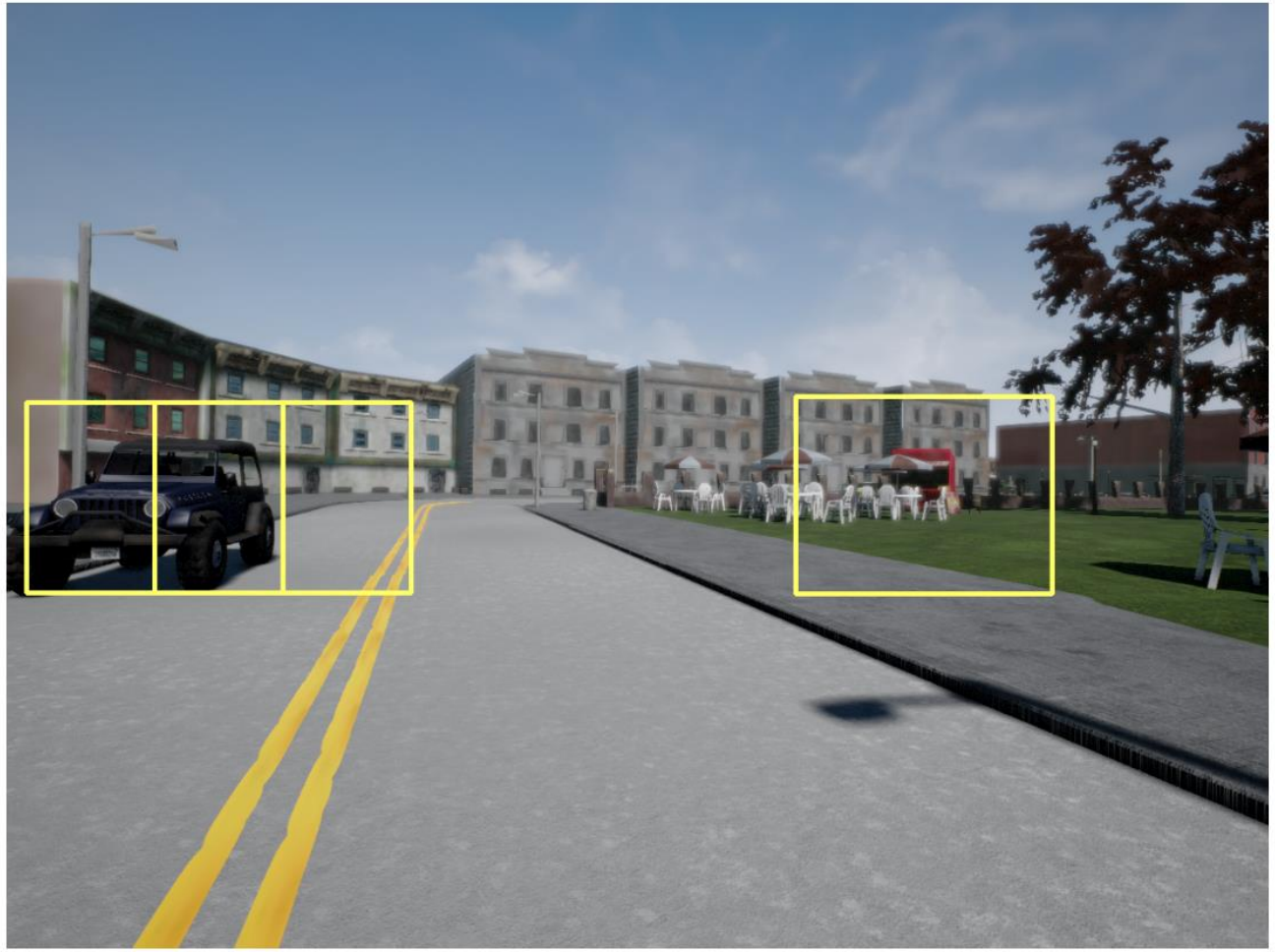
Object Detection

Solution: Use semantic segmentation to verify detections

Algorithm:

- For each detection bounding box: Count pixels belonging to predicted category Compute ratio = pixel count / bounding box area
- If ratio > threshold (0.3), keep detection
- Otherwise, discard as false positive

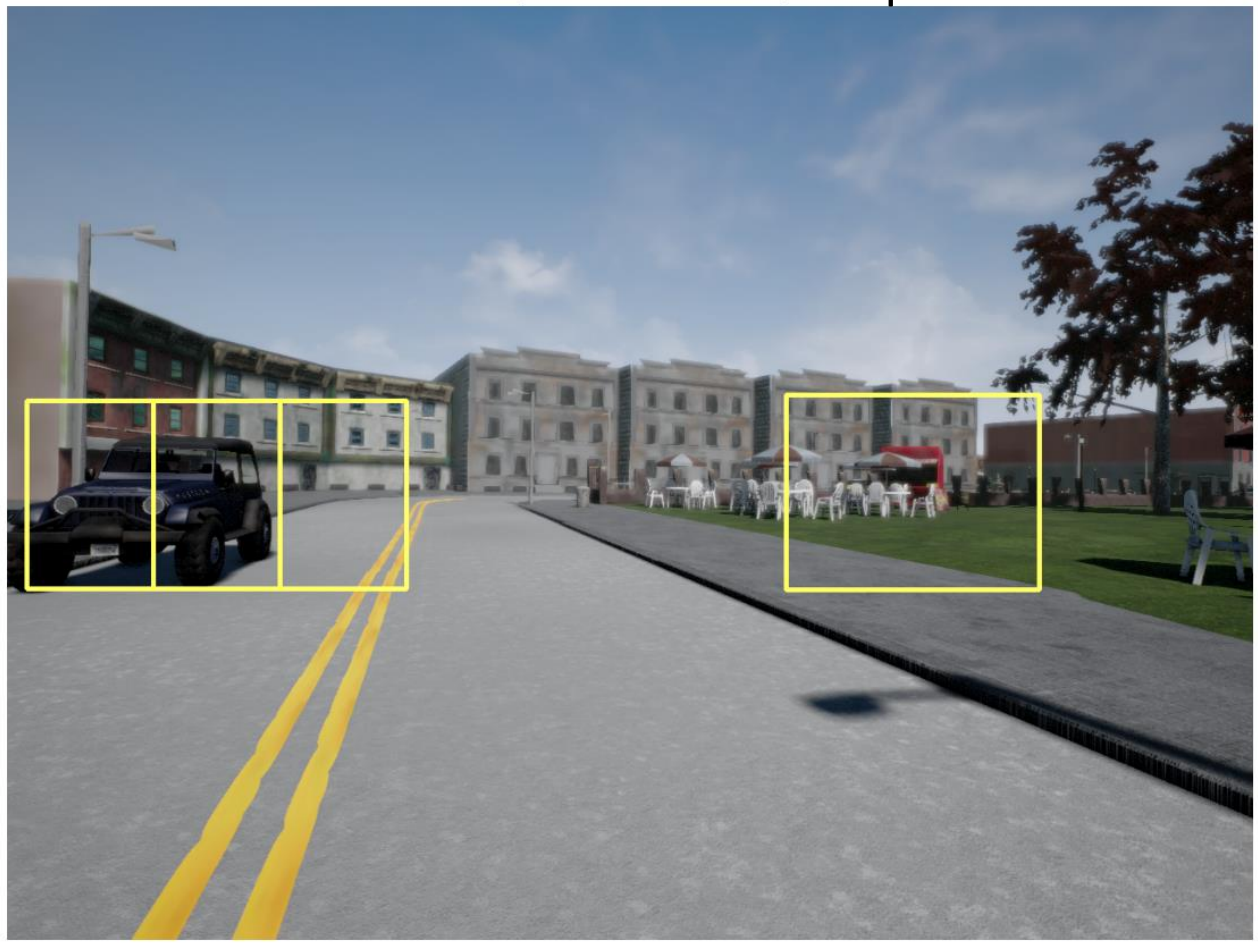
2D Object Detection Output



Object Detection

Category Mappings: Car/Cyclist → Segmentation index 10 (Vehicles)
Pedestrian → Segmentation index 4 (Pedestrians) (Note for other images in tutorial)

Raw detection network output



Filtered results

Task 3.
Distance Computation

Distance Computation

Now that the detected objects are shown on the image, the distance of the detected objects to the ego vehicle/camera can be calculated:

Algorithm:

- Compute distance to camera center for each pixel: $d = \sqrt{x^2 + y^2 + z^2}$
- For each bounding box, extract distance values
- Find minimum distance in bounding box
- This is the minimum distance to impact

Why minimum?

Conservative estimate for safety-critical applications